Spring IoC Container, Beans & Annotation-Based Configuration

Topics Covered in This PDF

1. Classes should not create their own dependencies
2. Dependency Injection and IoC
3. First-principles object creation, wiring and lifecycle
4. Maven project setup and `spring-context`
5. Spring Beans
6. Annotation-based vs XML-based configuration
7. `Student.class` and reflection metadata
8. `@Component`
9. `ApplicationContext`
10. `AnnotationConfigApplicationContext`
11. `AppConfig.class`
12. `@Configuration`
13. `@ComponentScan`
14. `getBean()`
15. Constructor, Field and Setter Injection
16. Why Constructor Injection is preferred
17. Spring startup internals
18. BeanDefinition
19. Dependency resolution
20. Missing dependency
21. Multiple matching beans
22. `@Primary`
23. `@Qualifier`
24. Default and custom bean names
25. `@Qualifier` with constructor, field and setter injection
26. `@Bean`
27. External library objects
28. Custom object creation

29. Custom names with `@Bean`
30. `@Bean` with dependencies
31. `@Component` vs `@Bean`
32. Avoid registering the same bean twice
33. `@Bean` dependency resolution
34. Why `main()` should not manage objects
35. `BeanFactory` vs `ApplicationContext`
36. Complete Spring startup flow
37. Key interview takeaways
38. Quick revision
39. 30-second recall flow

---

1. Core Idea — Classes Should Not Create Their Own Dependencies

A class should focus on its own responsibility.

Bad design:

```
public class OrderService {
    private PaymentService paymentService = new PaymentService();

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Flow:

```
OrderService
     |
     | creates
     v
PaymentService
```

`OrderService` is tightly coupled to a concrete `PaymentService`. If the implementation changes, `OrderService` must change.

Better:

```
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now the dependency is supplied from outside.

> **Dependency Injection = a class receives what it needs instead of creating it itself.**

---

2. First-Principles Understanding of IoC

Every application needs objects, and objects may need other objects.

Three questions always exist:

```
Who creates objects?
Who connects them?
Who manages their lifecycle?
```

Without Spring

```
main()
  |
  +--> creates objects
  +--> connects objects
  +--> manages setup
```

`main()` behaves like a small manual container.

With Spring

```
Spring IoC Container
  |
  +--> creates objects
  +--> connects objects
  +--> manages lifecycle
```

This is **Inversion of Control**: control of object creation and dependency wiring moves from application code to the container.

---

3. Basic Spring Core Setup

For annotation-based Spring Core:

1. Create a Maven project.
2. Add the `spring-context` dependency.

`spring-context` provides important container features such as:

- `ApplicationContext`
- Annotation-based configuration
- Component scanning
- Bean creation
- Dependency injection

Conceptually:

```
Maven project
     |
     v
spring-context
     |
     +--> container
     +--> scanning
     +--> beans
     +--> DI
```

4. What Is a Spring Bean?

> **A Spring Bean is an object managed by the Spring IoC Container.**

Spring manages:

```
Creation
   +
Dependency wiring
   +
Lifecycle
   +
Providing the object
```

So:

```
Normal Java object
        |
        | created + managed by Spring
        v
Spring Bean
```

5. Spring Configuration Styles

The PDF covers two styles.

Annotation-Based Configuration

Modern/common approach:

```
@Component
@Configuration
@ComponentScan
@Autowired
@Bean
```

XML-Based Configuration

Older Spring applications often used XML.

It is less preferred for new projects, but important for understanding legacy applications.

---

6. Why `Student.class` Matters

```
Class<Student> c = Student.class;
```

This does **not** create a `Student` object.

`Student.class` refers to a `Class` object containing metadata:

```
Student.class
   |
   +--> class name
   +--> fields
   +--> methods
   +--> constructors
   +--> annotations
```

Example:

```
Class name  -> Student
Fields      -> name, age
Methods     -> study()
Constructor -> Student()
Annotations -> @Component, @Service, ...
```

Spring uses class metadata internally.

Similarly:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

passes metadata about `AppConfig` so Spring can read its configuration instructions.

---

7. `@Component`

Spring does not automatically manage every class.

Use:

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment done");
    }
}
```

`@Component` means:

> This class is eligible to become a Spring Bean.

But Spring must also know **where to search**. That is the purpose of `@ComponentScan`.

---

8. `ApplicationContext`

Typical code:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

`ApplicationContext` represents the Spring IoC Container in normal applications.

Responsibilities include:

- Reading configuration
- Creating beans
- Resolving dependencies
- Managing bean lifecycle
- Providing beans when requested

This is invalid:

```
new ApplicationContext();
```

because `ApplicationContext` is an interface.

Use an implementation such as:

```
AnnotationConfigApplicationContext
```

---

9. `AnnotationConfigApplicationContext`

It is an implementation of `ApplicationContext` used to start a container with Java annotation configuration.

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

Meaning:

```
Start container
      ↓
Read AppConfig.class
      ↓
Process annotations
      ↓
Create/manage beans
```

10. `AppConfig.class`

Example:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {
}
```

It tells Spring:

- This is configuration.
- Scan `in.coderarmy`.
- Find component classes.
- Create their beans.
- Wire their dependencies.

Think:

> `AppConfig.class` **= Spring startup instructions.**

11. `@Configuration`

```
@Configuration
public class AppConfig {
}
```

Marks a class as a source of Spring configuration/bean definitions.

It may contain:

```
@ComponentScan
@Bean methods
Other configuration
```

---

12. `@ComponentScan`

```
@Configuration
@ComponentScan("com.coderarmy")
public class AppConfig {
}
```

Spring scans:

```
com.coderarmy
      |
      +--> sub-packages
```

It searches for:

```
@Component
@Service
@Repository
@Controller
```

and registers eligible classes as beans.

`@Component` vs `@ComponentScan`

```
@Component
    -> WHAT class can become a bean?

@ComponentScan
    -> WHERE should Spring search?
```

Without a package name:

```
@ComponentScan
```

Spring scans the package containing `AppConfig` and its sub-packages.

---

13. `getBean()`

After Spring creates and stores beans:

```
OrderService orderService =
    context.getBean(OrderService.class);
```

Meaning:

> Spring, give me the `OrderService` bean.

Complete example:

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        OrderService orderService =
            context.getBean(OrderService.class);

        orderService.placeOrder();
    }
}
```

If Spring does not know `OrderService`, common reasons are:

- Missing `@Component`
- Package not included in `@ComponentScan`
- Not registered with `@Bean` or XML

---

14. Types of Dependency Injection in Spring

The PDF covers:

```
Dependency Injection
   |
   +--> Constructor Injection
   +--> Field Injection
   +--> Setter Injection
```

---

15. Constructor Injection

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Spring supplies `PaymentService` through the constructor.

`@Autowired` on one constructor

In modern Spring, if a bean has only one constructor, `@Autowired` is optional.

Both are valid:

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

and:

```
@Autowired
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

---

16. Why Constructor Injection Is Preferred

1. Required dependency is explicit

If `OrderService` cannot work without `PaymentService`, the constructor makes that clear.

2. Dependency can be `final`

```
private final PaymentService paymentService;
```

It cannot accidentally be replaced after construction.

3. Complete object

The dependency is supplied while creating the object.

```
Create OrderService
       |
       +--> PaymentService already supplied
       |
       v
Complete usable object
```

4. Easy testing without Spring

```
PaymentService paymentService = new PaymentService();
OrderService orderService = new OrderService(paymentService);

orderService.placeOrder();
```

No Spring container is needed just to construct the class.

---

17. Field Injection

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Spring can use reflection to set the field.

Why it is generally not preferred

- Dependency is hidden.
- Harder to test without Spring.
- Field cannot be `final`.
- Object can exist in an incomplete state before injection.

> For required dependencies, constructor injection is generally preferred.

---

18. Setter Injection

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(
            PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Spring's conceptual flow:

```
Create OrderService
      ↓
Call setPaymentService()
      ↓
Pass PaymentService
      ↓
Dependency injected
```

Useful when the dependency:

- Is optional
- Can be changed after object creation

For mandatory dependencies, constructor injection is usually preferred.

---

19. Injection Comparison

Injection	Mechanism	Best use
Constructor	Constructor argument	Mandatory dependency
Setter	Setter method	Optional/changeable dependency
Field	Direct field injection	Works, but generally discouraged

Recall:

```
Mandatory  -> Constructor
Optional   -> Setter
Field      -> Generally avoid for clean design
```

---

20. What Happens Internally When Spring Starts?

When:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

runs, Spring conceptually performs:

```
1. Start IoC container
        ↓
2. Read AppConfig.class
        ↓
3. Process @Configuration
        ↓
4. Process @ComponentScan
        ↓
5. Scan package + sub-packages
        ↓
6. Find @Component/@Service/@Repository/@Controller
        ↓
7. Create BeanDefinitions
        ↓
8. Resolve dependencies
        ↓
9. Create bean objects
        ↓
10. Inject dependencies
        ↓
11. Store beans in container
        ↓
12. Return them through getBean()
```

This is one of the most important interview flows.

---

21. BeanDefinition

Before creating actual objects, Spring stores metadata describing them.

Example:

```
Bean name       -> paymentService
Bean class      -> PaymentService
Scope           -> singleton
Dependencies    -> none
Creation        -> constructor
```

This metadata is called:

> **BeanDefinition**

Important:

```
BeanDefinition != actual object
```

Think:

```
BeanDefinition
   |
   | instructions/metadata
   v
How Spring should create/manage bean

Bean
   |
   | actual object
   v
Object managed by Spring
```

---

22. Why BeanDefinitions Are Created First

Spring manages an entire application.

It first needs to know:

- Which beans exist
- Their classes
- Their scopes
- Their dependencies
- How to create them
- Lifecycle information

Conceptually:

```
BeanDefinition list
   |
   +--> paymentService
   +--> orderService
   +--> ...
```

Only after understanding this metadata does Spring create and wire actual objects.

---

23. Bean Creation and Dependency Resolution

If `PaymentService` has no dependency, Spring can conceptually do:

```
PaymentService paymentService =
    new PaymentService();
```

Internally Spring creates it.

Then for:

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

```
OrderService requires PaymentService
             |
             v
Search container for PaymentService
```

If found:

```
new OrderService(paymentService);
```

The dependency is injected during construction.

---

24. Dependency Resolution Flow

```
OrderService needs PaymentService
             |
             v
Does container have matching bean?
        /                        YES                  NO
       |                    |
       v                    v
inject it                error
```

After constructor injection:

```
OrderService
      |
      +--> PaymentService
```

So `OrderService` is ready to use.

---

25. Missing Dependency

If `OrderService` needs `PaymentService` but no matching bean exists, Spring cannot create `OrderService`.

Typical error:

```
No qualifying bean of type 'PaymentService' available
```

Common causes:

1. `PaymentService` is not annotated with `@Component`.
2. Its package is outside `@ComponentScan`.
3. No `@Bean` method creates it.
4. The dependency was not registered in Spring.

---

26. Multiple Matching Beans

Suppose:

```
public interface PaymentService {
    void pay();
}
```

Two beans:

```
@Component
public class UPIPaymentService implements PaymentService {
    public void pay() {
        System.out.println("UPI payment done");
    }
}
```

```
@Component
public class CardPaymentService implements PaymentService {
    public void pay() {
        System.out.println("Card payment done");
    }
}
```

And:

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

```
PaymentService
    |
    +--> UPIPaymentService
    |
    +--> CardPaymentService
```

Both match.

Spring cannot know which one to inject.

This is **bean ambiguity**.

---

27. `@Primary`

Use `@Primary` when one implementation should be the default.

```
@Primary
@Component
public class UPIPaymentService
        implements PaymentService {
}
```

Meaning:

> When multiple matching beans exist, prefer this bean by default.

28. `@Qualifier`

Use `@Qualifier` for an explicit choice.

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(
            @Qualifier("cardPaymentService")
            PaymentService paymentService) {

        this.paymentService = paymentService;
    }
}
```

Meaning:

> Inject the bean named `cardPaymentService`.

Memory:

```
@Primary
    -> default choice

@Qualifier
    -> specific choice
```

If both are used:

> **`@Qualifier` gets priority over `@Primary`.**

Specific choice beats default choice.

29. Default Bean Names

Spring commonly derives a component bean name from the class name.

Example:

```
CardPaymentService
        ↓
cardPaymentService
```

Acronym-style class names can follow Java bean naming rules differently, so explicit names are useful when names matter.

---

30. Custom Bean Names with `@Component`

```
@Component("upi")
public class UPIPaymentService
        implements PaymentService {
}
```

```
@Component("card")
public class CardPaymentService
        implements PaymentService {
}
```

Now:

```
public OrderService(
        @Qualifier("card")
        PaymentService paymentService) {

    this.paymentService = paymentService;
}
```

This is clearer and easier to control.

---

31. `@Qualifier` with Field Injection

```
@Component
public class OrderService {

    @Autowired
    @Qualifier("upiPaymentService")
    private PaymentService paymentService;
}
```

This works, but constructor injection is still generally preferred.

---

32. `@Qualifier` with Setter Injection

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(
            @Qualifier("cardPaymentService")
            PaymentService paymentService) {

        this.paymentService = paymentService;
    }
}
```

Spring injects the specified bean through the setter.

---

33. Why Do We Need `@Bean`?

`@Component` works well when we own the class.

But suppose an external library provides:

```
public class EmailClient {

    private final String apiKey;

    public EmailClient(String apiKey) {
        this.apiKey = apiKey;
    }

    public void sendEmail() {
        System.out.println(
            "Email sent using API key: " + apiKey
        );
    }
}
```

We normally cannot modify the external class to add:

```
@Component
```

But we still want Spring to manage an `EmailClient`.

Use:

```
@Bean
```

---

34. Another Use Case for `@Bean` — Custom Creation

Example:

```
public class User {

    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }
}
```

Spring may not know what values to supply for `name` and `email`.

`@Bean` lets us explicitly define how the object should be created.

---

35. What Exactly Does `@Bean` Mean?

`@Bean` is placed on a method inside a configuration class.

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Conceptual flow:

```
@Bean method
      ↓
Spring calls method
      ↓
Method returns object
      ↓
Spring registers object
      ↓
Spring manages it as a bean
```

For singleton beans, Spring normally creates and stores the object when the context starts.

---

36. Custom Bean Name with `@Bean`

Default bean name = method name.

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

Name:

```
paymentService
```

Custom name:

```
@Bean("myPaymentService")
public PaymentService paymentService() {
    return new PaymentService();
}
```

Retrieve by name:

```
PaymentService paymentService =
    (PaymentService)
    context.getBean("myPaymentService");
```

---

37. `@Bean` with Dependencies

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService(
            PaymentService paymentService) {

        return new OrderService(paymentService);
    }
}
```

Flow:

```
paymentService()
      |
      v
PaymentService bean
      |
      v
orderService(PaymentService)
      |
      v
OrderService bean
```

Spring resolves the dependency and passes the `PaymentService` bean into the `orderService()` method.

An alternative is:

```
@Bean
public OrderService orderService() {
    return new OrderService(paymentService());
}
```

In a full `@Configuration` class, Spring manages singleton behavior for such configuration method calls. The parameter-based style is generally cleaner and easier to understand.

---

38. `@Component` vs `@Bean`

Point	`@Component`	`@Bean`
Applied to	Class	Method
Registration	Automatic detection	Manual registration
Best for	Our own classes	External/custom objects
Component scanning	Yes	Not for the bean itself
Default bean name	Class-name based	Method name

Memory

```
@Component
    = "Spring, find this class."

@Bean
    = "Spring, use this method to create this object."
```

---

39. Avoid Registering the Same Bean Twice

Avoid doing both:

```
@Component
public class PaymentService {
}
```

and:

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

unless multiple beans are intentional.

Possible problems:

- Two beans may exist.
- Injection by type can become ambiguous.
- Same-name registration can involve bean overriding rules depending on configuration.

Clean rule:

> **Use either `@Component` or `@Bean` for a bean unless you intentionally need multiple beans.**

---

40. Dependency Resolution with `@Bean`

The same rules apply to `@Bean`-created beans.

Default choice:

```
@Bean
@Primary
public PaymentService upiPaymentService() {
    return new UPIPaymentService();
}
```

Specific choice:

```
@Bean
public OrderService orderService(
        @Qualifier("cardPaymentService")
        PaymentService paymentService) {

    return new OrderService(paymentService);
}
```

---

41. Why Not Put Everything in `main()`?

If `main()` creates every object:

```
public static void main(String[] args) {

    PaymentService paymentService =
        new PaymentService();

    OrderService orderService =
        new OrderService(paymentService);

    // many more objects...
}
```

`main()` becomes responsible for:

- Object creation
- Wiring
- Configuration
- Object management

This defeats the purpose of the IoC container.

Better:

```
public static void main(String[] args) {

    ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);

    OrderService orderService =
        context.getBean(OrderService.class);

    orderService.placeOrder();
}
```

> `main()` **should start the application, not manage the entire object graph.**

---

42. BeanFactory vs ApplicationContext

Spring has `BeanFactory`, the basic container interface.

`ApplicationContext` builds on top of it and provides more complete features.

```
BeanFactory
    ↓
Basic container

ApplicationContext
    ↓
Advanced/common application container
```

`ApplicationContext` provides features such as:

- Bean creation and DI
- Lifecycle management
- Event publishing
- Internationalization
- Integration with other Spring features

---

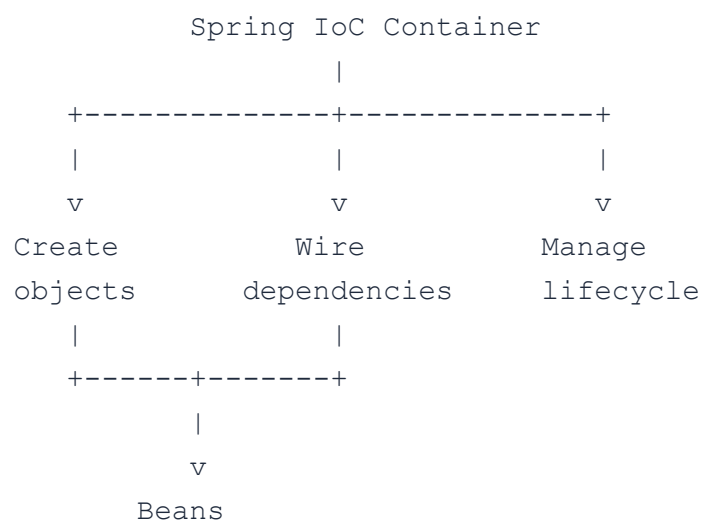
43. Complete Spring Startup Flow

```
Application starts
      |
      v
new AnnotationConfigApplicationContext(AppConfig.class)
      |
      v
Create ApplicationContext
      |
      v
Read AppConfig.class
      |
      v
Process @Configuration
      |
      v
Process @ComponentScan
      |
      v
Scan package + sub-packages
      |
      v
Find @Component / @Service / @Repository / @Controller
      |
      v
Create BeanDefinitions
      |
      v
Resolve dependencies
      |
      v
Create bean objects
      |
      v
Inject dependencies
      |
      v
Store beans in container
      |
      v
context.getBean(...)
      |
```

```
      v
Return ready-to-use bean
```

---

44. Most Important Mental Model



---

45. High-Value Interview Questions

Q1. What is a Spring Bean?

An object whose creation, dependency wiring, and lifecycle are managed by the Spring IoC container.

Q2. What is IoC?

Inversion of Control means control of object creation and dependency wiring moves from application code to an external container.

Q3. What is Dependency Injection?

A class receives its dependencies from outside instead of creating them itself.

Q4. What is `ApplicationContext`?

The commonly used Spring IoC container interface that creates/manages beans, resolves dependencies, manages lifecycle, and provides additional application features.

Q5. What is `AnnotationConfigApplicationContext`?

An `ApplicationContext` implementation used for annotation-based Java configuration.

Q6. What does `@Configuration` do?

Marks a class as a source of Spring configuration and bean definitions.

Q7. What does `@ComponentScan` do?

Tells Spring where to search for component classes.

Q8. What does `@Component` do?

Marks a class as eligible to become a Spring Bean.

Q9. What does `getBean()` do?

Asks the Spring container for a bean by type or name.

Q10. What is BeanDefinition?

Metadata describing how Spring should create and manage a bean. It is not the actual object.

Q11. Why constructor injection?

It makes required dependencies explicit, allows `final`, creates a complete object, and makes testing easier without Spring.

Q12. Is `@Autowired` required for one constructor?

No. Modern Spring can automatically use the single constructor.

Q13. Why is field injection discouraged?

The dependency is hidden, testing is harder, the field cannot be `final`, and the object can temporarily be incomplete.

Q14. When is setter injection useful?

For optional or changeable dependencies.

Q15. What happens if no matching bean exists?

Spring cannot resolve the dependency and reports a qualifying-bean error.

Q16. What if multiple beans match?

Use `@Primary` for a default or `@Qualifier` for a specific bean.

Q17. Which wins: `@Primary` or `@Qualifier`?

`@Qualifier` because it is the specific selection.

Q18. Why use `@Bean`?

For external classes or when we need explicit/custom object creation.

Q19. `@Component` vs `@Bean`?

`@Component` is class-level automatic detection; `@Bean` is method-level explicit registration.

Q20. Why not put everything in `main()`?

Because Spring should manage object creation, dependency wiring and lifecycle; `main()` should mainly start the application.

Q21. BeanFactory vs ApplicationContext?

BeanFactory is the basic container; ApplicationContext is the more complete container commonly used in applications.

---

46. QUICK REVISION — Notebook Recall

Core

> **Classes should focus on their responsibility, not create all their dependencies.**

```
Bad:
OrderService -> new PaymentService()

Better:
Outside/Spring -> PaymentService
                        ↓
                  OrderService
```

Bean

> **Spring-managed object = Spring Bean**

Spring manages:

```
Creation
Wiring
Lifecycle
Retrieval
```

Container

```
ApplicationContext
      ↓
Spring IoC Container
```

Configuration

```
@Configuration
    -> configuration class

@ComponentScan
    -> where to search

@Component
    -> class eligible for bean

@Bean
    -> method explicitly creates/registers bean
```

`getBean()`

```
context.getBean(OrderService.class);
```

> Ask Spring for the bean.

DI

```
Constructor -> mandatory
Setter      -> optional/changeable
Field       -> works, generally not preferred
```

Constructor Injection

Remember:

```
Required dependency
       +
final possible
       +
complete object
       +
easy testing
```

BeanDefinition

```
BeanDefinition = metadata
Bean           = actual object
```

Dependency Resolution

```
Need dependency
      ↓
Find matching bean
   /        YES        NO
  ↓          ↓
inject      error
```

Multiple Beans

```
@Primary
   -> default

@Qualifier
   -> specific

Qualifier > Primary
```

`@Bean`

Use for:

```
External library
Custom construction
Explicit configuration
```

`@Component` vs `@Bean`

```
@Component -> class-level + automatic discovery

@Bean      -> method-level + manual registration
```

Bean Names

```
@Component("card")
        -> card

@Bean("myPaymentService")
        -> myPaymentService

@Bean method name
        -> default bean name
```

Container Choice

```
BeanFactory
    -> basic

ApplicationContext
    -> advanced/common
```

---

47. 30-Second Interview Recall

Memorize this flow:

```
Application needs objects
        ↓
Objects need other objects
        ↓
Who creates + connects + manages them?
        ↓
Spring IoC Container
        ↓
ApplicationContext
        ↓
AppConfig.class
        ↓
@Configuration
        ↓
@ComponentScan
        ↓
Find components
        ↓
Create BeanDefinitions
        ↓
Resolve dependencies
        ↓
Create beans
        ↓
Inject dependencies
        ↓
Store beans
        ↓
getBean()
        ↓
Ready-to-use object
```

One-line final answer

> **Spring IoC Container creates and manages beans, resolves and injects their dependencies, manages their lifecycle, and provides the ready-to-use objects when requested.**